Report on Objective 1: PF Layer Buffer Manager Enhancements

1. Objective

The primary objective was to enhance the Paged File (PF) layer's buffer manager. The initial implementation provided a fixed-size buffer (PF_MAX_BUFS), a non-selectable LRU eviction policy, and a basic dirty page handling mechanism.
The goals were to:

1. Implement a configurable buffer pool size.
2. Introduce selectable page replacement strategies (LRU and MRU).
3. Integrate a statistics collection module for performance monitoring.
4. Analyze the performance trade-offs between the implemented strategies.

2. Implementation Summary

Modifications were concentrated within the PF layer files: pf.c, buf.c, pf.h, and pftypes.h.

2.1. Configurable Parameters

The PF_Init function signature in pf.h was modified to accept configuration parameters:
void PF_Init(int num_buffers, int replacement_strategy);
To support this, the hardcoded #define PF_MAX_BUFS was removed from pftypes.h, and strategy constants (STRATEGY_LRU, STRATEGY_MRU) were added. In buf.c, static global variables (PFmaxbpage, PFstrategy) were introduced to store these runtime values, and the buffer allocation logic was updated to use PFmaxbpage.

2.2. Page Replacement Logic

The existing implementation utilized a doubly linked list, implicitly functioning as an LRU policy by evicting from the tail (PFlastbpage).
We modified the victim selection logic in PFbufInternalAlloc to check the PFstrategy variable.

- **LRU:** The original behavior (evict from PFlastbpage) is maintained.
- **MRU:** A new logic path was added to scan from the head of the list (PFfirstbpage) and evict the *most* recently used non-fixed page.

2.3. Dirty Flag

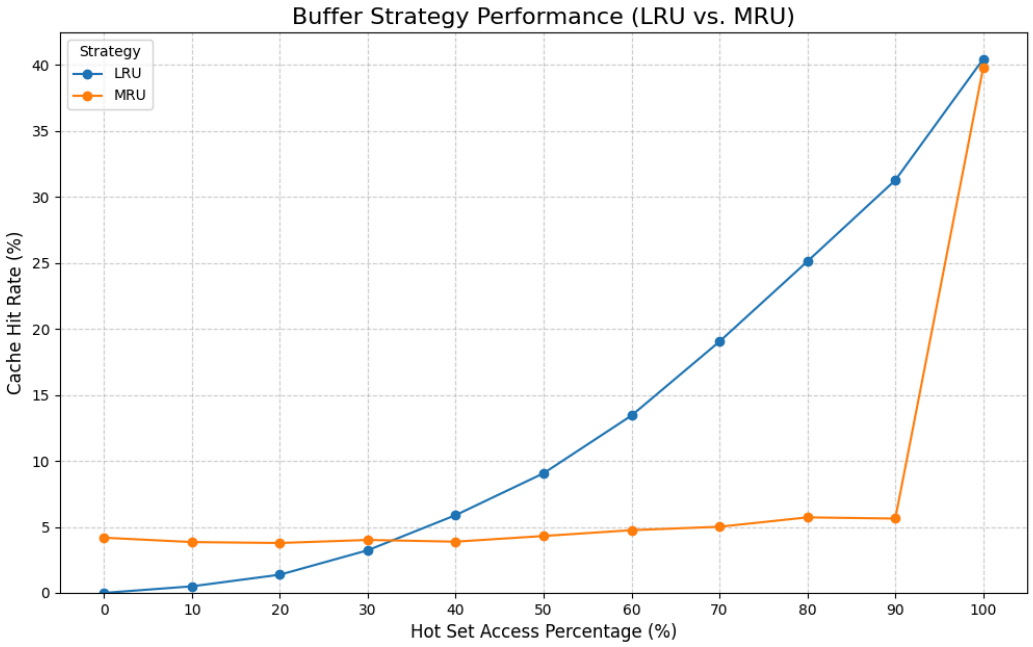
The existing dirty flag (dirty : 1;) and write-back policy were retained without modification. Dirty pages are correctly written to disk only upon eviction or via PF_CloseFile, ensuring data persistence.

2.4. Statistics Collection

A statistics module was integrated into buf.c to provide performance metrics.

1. **Counters:** Five static global counters were added (e.g., g_num_logical_reads, g_num_buffer_hits, g_num_physical_reads).
2. **Instrumentation:** Key functions were instrumented:
    - PFbufGet: Increments g_num_logical_reads on every call. It increments g_num_buffer_hits on a cache hit or g_num_physical_reads on a miss.
    - PFbufAlloc: Increments g_num_logical_writes.
    - PFbufInternalAlloc / PFbufReleaseFile: Increment g_num_physical_writes when a dirty page is flushed.
3. Verification: The implementation correctly satisfies the identity:
   Logical Reads = Buffer Hits + Physical Reads

3. Performance Analysis



A test program (buffertest.c) was developed to simulate a mixed workload, consisting of:

1. **Hot Set (50 pages):** Frequent, random access (80% of requests).
2. **Cold Set (450 pages):** Sequential access (20% of requests).
3. **Buffer Size (20 pages):** Sized to hold the hot set but not the entire cold set.
4. **Accesses per test :** 10000 Read access per test (point on graph)

3.1. Analysis of Results

LRU Dominance (Blue Line):

The Least Recently Used strategy shows a strong positive correlation with the "Hot Set

Access Percentage." As the system accesses the "hot" (frequently needed) data more often, the LRU hit rate climbs steadily, moving from nearly **0%** to a peak of roughly **40%**. This suggests LRU effectively retains the "hot" data in the cache.

MRU Stagnation (Orange Line):

The Most Recently Used strategy remains largely ineffective for the vast majority of the test cases. It hovers around a **4-5%** hit rate regardless of whether the hot set access is 0% or 90%.

The Anomaly at 100%:

There is a massive spike for MRU at the **100%** mark, where it suddenly matches the LRU performance. This indicates that MRU only becomes viable in this specific scenario when the workload is exclusively accessing the hot set, likely due to the cache size matching the hot set size exactly at that limit.

Crossover Point:

There is a crossover point between **20% and 30%**. Below this threshold, MRU actually performs marginally better than LRU, but as the workload focuses more on specific "hot" data, LRU rapidly overtakes it.